Entrega 4 — búsqueda, ordenamiento y complejidad

Vence: domingo 01-nov-2026 23:59 · **Peso:** 9 % · Tag: `entrega-4`

Objetivo: darle cerebro al sistema. Implementar (a mano) búsqueda lineal y binaria, dos ordenamientos propios, y justificar las cotas con una tabla de complejidad y mediciones reales con `time.perf_counter`. Prohibido `sorted`, `list.sort` y `bisect` para cumplir estos ítems (usarlos = N en ese ítem).

1. Mail

```
Para: diego.ambrossio@unab.edu.ar; angel.bianco@unab.edu.ar
Asunto: AyED C2 2026 Comision 1 - GRUPO 04 - Entrega 4

Repo: https://github.com/comision1-grupo4/ayed-2026-recetario
Entrega de E4: búsqueda lineal y binaria propias, dos ordenamientos
propios, tabla de complejidad y mediciones con time.perf_counter.
Tag: entrega-4.
```

2. Qué tiene que tener el repo

#	Ítem	Regla dura
4.1	Búsqueda lineal propia, usada desde el CLI	opción "buscar" del menú la usa
4.2	Búsqueda binaria propia sobre colección ordenada	el CLI no la ofrece si el catálogo no está ordenado (o reordena y avisa)
4.3	Dos ordenamientos propios, criterios distintos (o un criterio, estable vs. no estable)	sin `sorted` ni `list.sort`
4.4	Tabla de complejidad de las operaciones del sistema	`docs/INFORME.md` §5, con justificación breve
4.5	Medición con `time.perf_counter`	dataset original + copia agrandada (o 2 tamaños)
4.6	Tag corre; no rompieron E2/E3	regresión: recursión, TADs y listado intactos

3. Ejemplo corto

```python
def busqueda_binaria(coleccion, clave, obtener_clave):
    izq, der = 0, len(coleccion) - 1
    while izq <= der:
        medio = (izq + der) // 2
        valor = obtener_clave(coleccion[medio])
```

```
        if valor == clave:
            return coleccion[medio]
        if valor < clave:
            izq = medio + 1
        else:
            der = medio - 1
    return None   # no encontrado: el CLI avisa, no explota
```

Medición en `docs/INFORME.md` §5 — se nota que midieron, no hace falta un paper:

Operación	n	segundos
búsqueda binaria (título)	67	0.000012
búsqueda binaria (n × 10)	670	0.000014

4. Estructura sugerida del repo (creció desde E3)

```
ayed-2026-recetario/
├── README.md
├── src/
│   ├── config.py
│   ├── main.py               # + opciones buscar y ordenar
│   ├── excepciones.py
│   ├── dominio/              # intacto de E3
│   ├── algoritmos/           # E4: acá vive todo
│   │   ├── busqueda.py       # busqueda_lineal, busqueda_binaria
│   │   └── ordenamiento.py   # ordenar_seleccion, ordenar_insercion
│   ├── persistencia/
│   │   └── texto.py
│   └── tads/                 # intacto de E3
├── data/
└── docs/
    ├── INFORME.md            # + §5 complejidad y mediciones
    ├── PROTOCOLO_PRUEBAS.md  # + casos P09 a P12 ejecutados
    └── DECLARACION_IA.md     # + fila E4
```

5. Tag y regresión

```
git add . && git commit -m "E4: búsqueda, orden y complejidad"
git tag entrega-4
git push && git push origin entrega-4
```

Nota extra (hasta 2 puntos): un **tercer** algoritmo de ordenamiento medido en el informe, o usar la lista enlazada también en el catálogo **y** un índice `dict` comparado (qué ganás y qué pagás).